

毕业设计周记

第一周

【毕业设计进度情况（本周单元设计的进度情况）】

本周主要是确定毕业设计的题目和方向。课题定的是"局域网多源流媒体系统"，简单来说就是做一个能在局域网内采集屏幕、摄像头、本地文件然后推流，同时也能拉流播放的桌面客户端，外加一个服务端做中转，再配一个手机端播放器。这周花了不少时间调研市面上类似的产品，像OBS、ZLMediaKit这些，大致理清了要做的功能模块，也把主要的应用场景列了一下。反复权衡之后确定了三个子项目：桌面端、手机端、服务端，桌面端是核心。

【指导教师指导情况】

导师帮忙敲定了选题方向，建议我不要贪大求全，先把桌面端的核心功能做扎实，服务端和手机端可以作为扩展。还推荐了ZLMediaKit做流媒体分发内核，让我先跑通它的demo感受一下流程。

【遇到的疑难问题】

对整体技术栈还不够熟悉，尤其是FFmpeg的API体系太庞大了，光是看文档就有点头大，各种结构体和函数指针的关系绕来绕去。C++17和Qt6的组合以前没怎么系统用过，心里有点没底，怕踩坑太多影响进度。

【疑难问题的解决途径】

决定先不急着写代码，花几天时间把FFmpeg官方examples逐个跑一遍，重点看编码、推流、拉流相关的demo，边跑边做笔记。Qt这边打算跟着官方文档做个小项目热热身，先熟悉一下信号槽和多线程的用法再正式动工。

第二周

【毕业设计进度情况（本周单元设计的进度情况）】

这周正式搭了项目骨架。桌面端选型确定了C++17 + Qt 6.10.2 + CMake 3.15以上，用FFmpeg 8.x做媒体后端。画了一个七层分层架构图：app入口层 -> widget界面层 -> service业务层 -> runtime工厂层 -> backend实现层 -> api接口层 -> common工具层。明确了"上层只依赖接口，不感知具体实现"的原则，每层之间通过接口解耦。手机端暂定Flutter，服务端用Go写网关。

【指导教师指导情况】

导师看了架构图后提了几个意见：一是runtime层用工厂模式来隔离不同后端实现（FFmpeg和Qt6）是个好思路；二是建议frame bus用Qt的COW机制做零拷贝，避免大量YUV帧数据来回复制，这个建议后来确实成了性能优化的关键。

【遇到的疑难问题】

七层架构听起来挺好，但实际写接口的时候发现各层之间的依赖关系比想象中复杂不少。比如屏幕采集的帧怎么高效传给编码器，中间要不要加缓冲队列，缓冲多大才不会丢帧，这些细节之前在画图的时候没考虑太细。

【疑难问题的解决途径】

参考了OBS的架构设计，他们用了全局的帧总线来解耦采集和编码，思路很巧妙。我借鉴这个思路设计了一个FrameBus单例，用信号槽的方式发布帧数据，各模块按需订阅即可，完全不用直接耦合，后续加新模块也方便。

第三周

【毕业设计进度情况（本周单元设计的进度情况）】

这周主要搞定构建系统。CMake这边配好了AUTOMOC、AUTORCC、AUTOUIIC，引入了CPM做第三方依赖管理，把spdlog、yaml-tool都拉进来了，省得手动下载配置。最重要的是搞定了FFmpeg 8.x的预编译库集成，放在3rd目录下，写了cmake脚本自动查找和链接，区分了Debug和Release两套库。还加上了CPack打包支持，Windows下面用NSIS打exe安装包，Linux用DEB/TGZ，顺便配了windeployqt自动拷贝Qt的dll。

【指导教师指导情况】

导师在群里分享了一些CMake最佳实践的文章，提醒我注意把public和private的链接属性分清楚，不然后面编译依赖会乱。另外建议尽早把CI流程搭起来，每次提交自动编译能省很多排查时间。

【遇到的疑难问题】

FFmpeg 8.x在Windows上用MSVC编译预编译库的时候，发现有些头文件路径不对，链接的时候报了LNK2019。折腾了好久才搞清楚是avformat和avcodec的版本对应关系有问题，两个库的小版本号没对上。

【疑难问题的解决途径】

最后发现是3rd目录下放的FFmpeg版本跟CMake脚本里写的版本号不一致，重新对齐版本号就好了。顺便写了个检测脚本，以后换版本的时候先跑一下确认所有库的版本匹配，免得再踩同样的坑。

第四周

【毕业设计进度情况（本周单元设计的进度情况）】

这周开始写界面。用Qt Designer画了CaptureWindow的主界面布局：上面是视频预览区，左边是模式切换按钮（摄像头/文件/屏幕/组合），下面是控制栏（推流/拉流/截图/录制），整体布局尽量简洁直观。实现了深色和浅色主题切换，定义了一套DesignTokens管理颜色和间距，方便后面统一调整。FVideoView这个自定义OpenGL控件也搭了个架子，继承QOpenGLWidget，准备后面用来渲染视频帧。

【指导教师指导情况】

导师看了界面后建议把推流和拉流的控制做成弹窗而不是直接堆在主界面上，因为参数比较多（应用名、流名、协议选择等），直接放主界面太拥挤。这个建议很实用，后续改成了弹窗形式。

【遇到的疑难问题】

Qt Designer生成的ui文件配合CMake的AUTOUIC自动处理，但是自定义控件FVideoView在Designer里没法直接预览，只能提升一个QWidget占位，在设计阶段看不到效果只能脑补布局，有点不方便。

【疑难问题的解决途径】

查了Qt文档发现可以通过写一个Designer插件来让FVideoView在设计时预览，但考虑到时间成本和当前阶段的优先级，决定先用QWidget占位的方式凑合着用，后期如果有空余时间再回来补这个插件。

第五周

【毕业设计进度情况（本周单元设计的进度情况）】

这周重点写api接口层和runtime工厂层。把五个核心接口定义完了：ICamera（摄像头采集）、IPlayer（文件播放）、IScreenCapture（屏幕采集）、IStream（推拉流）、IFVideoView（视频渲染预览）。每个接口只暴露必要的虚函数，实现细节全扔到backend里，保证接口干净。Runtime工厂类根据MediaBackendType枚举来创建对应的实现，上层调用完全感知不到底层用的是FFmpeg还是Qt6，做到了真正的实现无关。

【指导教师指导情况】

导师review了接口设计，指出IPlayer和ICamera有些方法可以抽到一个公共基类里，毕竟都是"媒体源"这个概念。不过我们讨论后还是保持独立接口，因为它们的操作语义差异挺大，强行合并反而让调用方困惑。

【遇到的疑难问题】

工厂模式返回shared_ptr的时候，发现Qt的信号槽跟多态一起用容易踩坑——如果基类没有继承QObject，派生类继承了QObject，信号槽就不好用了，编译能过但运行时连接不上。最后决定让接口类也继承QObject保持一致性。

【疑难问题的解决途径】

让所有Api接口类都继承QObject，用Q_DECLARE_METACLASS和qobject_cast来做安全的向下转型。虽然多了一点MOC的编译开销，但换来了类型安全和信号槽的便利，跟Qt框架的契合度也更好，整体值得。

第六周

【毕业设计进度情况（本周单元设计的进度情况）】

这周开始搞摄像头采集模块。实现了两套后端：media_ffmpeg里用libavdevice的dshow（Windows）和v4l2（Linux）来采集摄像头；media_qt6里用QCamera和QVideoSink来采集。两套实现都遵循ICamera接口，支持列出摄像头列表、切换分辨率帧率、暂停恢复。还写了一个CameraFrameBus——线程安全的YUV帧总线，采集线程往里写，预览和编码线程往外读，用Qt的COW机制避免了深拷贝，内存开销很小。

【指导教师指导情况】

导师提醒我注意摄像头设备独占的问题——如果预览和编码同时打开同一个摄像头设备，Windows上可能会报设备占用。建议我用帧总线做中转，预览和编码都从总线取帧，而不是各自打开设备。

【遇到的疑难问题】

QCamera的QVideoSink回调里拿到的QVideoFrame，转成YUV420P格式的时候发现有些摄像头的原生格式是NV12或者MJPG，需要用FFmpeg的sws_scale做转换。但转换性能不太理想，1080p@30fps能掉到20fps左右，预览有明显卡顿。

【疑难问题的解决途径】

做了两方面的优化：一是加了格式缓存，同一个摄像头记住它支持的最优格式，避免每次都重新枚举格式列表；二是sws_scale的flags设成SWS_FAST_BILINEAR，换掉了默认的SWS_BICUBIC，牺牲一点画质换性能，帧率稳回30fps。

第七周

【毕业设计进度情况（本周单元设计的进度情况）】

这周写视频渲染模块。FVideoWidget从QOpenGLWidget继承，用OpenGL的PBO异步上传纹理，双缓冲机制降低渲染延迟。当摄像头帧通过FrameBus发过来时，先把YUV数据拷贝到PBO，在下一帧的paintGL里直接绑定纹理渲染，减少GPU空闲等待。还搞了一个PreviewTarget结构体，把widget指针和surface格式打包在一起，让runtime层做绑定的时候更干净。同时把文件播放也接上了——media_ffmpeg里用avformat打开文件，解码后用同样的FrameBus统一发帧到渲染控件。

【指导教师指导情况】

导师建议测试一下不同分辨率下渲染的帧率表现，特别要注意4K视频的解码+渲染会不会掉帧。另外提醒PBO的大小要跟实际帧大小匹配，不然驱动可能会做额外的内部内存拷贝。

【遇到的疑难问题】

渲染的时候发现画面有撕裂，特别是在高帧率场景下，屏幕中间明显有一条水平分界线。一开始以为是OpenGL没开vsync，但检查后发现Qt的QSurfaceFormat已经设了DoubleBuffer和vsync，问题可能出在帧同步上。

【疑难问题的解决途径】

最后定位到是FrameBus的读写锁粒度太大，渲染线程经常拿不到最新帧就在等锁。把锁改成了读写锁+双缓冲——采集线程写备用缓冲，渲染线程读当前缓冲，写完交换指针，几乎没锁竞争，画面撕裂彻底消失了。

第八周

【毕业设计进度情况（本周单元设计的进度情况）】

这周搞屏幕采集，这是整个项目里最平台相关的模块，不同操作系统的方案差异很大。Windows上实现了两套方案：desktopcapture_dxgi用DXGI Desktop Duplication API，性能最好，能抓到桌面变化区域的纹理；media_ffmpeg里用gdigrab设备做备选。Linux上用x11grab。都实现了IScreenCapture接口，支持列出屏幕、选择采集区域、控制帧率、是否采集光标等。ScreenFrameBus跟CameraFrameBus设计类似，也是COW零拷贝。

【指导教师指导情况】

导师帮我看了DXGI的代码，指出AcquireNextFrame的超时时间设得太长（500ms），在屏幕没变化的时候会卡很久导致帧率不稳定，建议改成50ms以内然后快速重试，这样整体响应更灵敏。

【遇到的疑难问题】

DXGI Desktop Duplication在笔记本双显卡环境下抓不到独立显卡渲染的画面，只能抓到集成显卡的输出。这对游戏画面或者GPU加速应用的内容采集是个大问题，抓出来是黑屏或者静止画面。

【疑难问题的解决途径】

查了微软文档发现这是DXGI的设计限制——Desktop Duplication只能抓取当前桌面渲染所在的GPU。目前没有完美的跨GPU解决方案，折中方案是优先用集成显卡运行桌面，或者在用户文档里明确标注这个限制。

第九周

【毕业设计进度情况（本周单元设计的进度情况）】

这周是整个项目最难啃的骨头——stream_ffmpeg推拉流引擎。核心类StreamFFmpeg管理推流和拉流两个独立线程，各自维护自己的AVFormatContext。推流端实现了remuxLoop（不重新编码，直接换封装格式推RTMP）和transcodeFileLoop（重新编码，支持调码率、分辨率、帧率）。用avformat_alloc_output_context2创建输出上下文，avio_open连RTMP服务器，然后循环写包。拉流端用avformat_open_input打开远端流，解码后通过FrameBus发到渲染层。

【指导教师指导情况】

导师看了推流代码后强调编码器预设的选择很关键——如果目标是低延迟直播，h264_nvenc和h264_amf的preset要设成p1（最快），而不是默认的medium。还建议把编码参数做成可配置的，不要写死。

【遇到的疑难问题】

推流到ZLMediaKit的时候，偶尔会出现连接断开后无法自动重连的情况，大概每二十分钟就会遇到一次。avio_open返回成功，但写了几帧后就报broken pipe，怀疑是网络波动或者ZLM那边超时踢掉了连接。

【疑难问题的解决途径】

加了一个重连机制：推流线程检测到写包失败后，先调用avio_close清理当前连接，等2秒再重新avio_open，最多重试3次。同时在ZLM的配置文件里把RTMP超时时间从默认的10秒调到了30秒，两边配合后稳定性明显改善。

第十周

【毕业设计进度情况（本周单元设计的进度情况）】

这周继续深入推流引擎，完善了各种场景的推流入口。实现了三种场景推流：pushScreenLoop直接从屏幕采集+编码推流；pushCameraLoop从摄像头采集+编码推流；pushScreenPreviewLoop和pushCameraPreviewLoop分别从预览帧推流，避免设备重复打开导致占用冲突。音频这边搞了Windows WASAPI的loopback采集，能把系统声音也一起推出去，不用依赖虚拟声卡。还加了拉流录制功能——拉流的同时用独立的编码管道把流存成MP4文件，录制时长实时显示在界面上。

【指导教师指导情况】

导师帮我审了音频采集模块的代码，发现WASAPI的buffer大小设得有点小（10ms），在高负载下容易丢帧爆音。建议调到20-30ms，虽然延迟稍微大一点但稳定性好很多。

【遇到的疑难问题】

推流的时候音视频同步是个大坑。视频用了系统时钟的PTS，音频用了另一个时钟源，两边的基准不一样。推到服务器后播放端出现口型对不上的情况，大概差了200-300ms，看着非常难受。

【疑难问题的解决途径】

统一了时间基准——推流开始前用av_gettime_relative获取一个基准时间戳，音视频帧的PTS都基于这个基准来计算偏移，确保用同一个时钟源。调试的时候打了不少PTS日志来验证对齐效果，反复调了几轮终于对上了。

第十一周

【毕业设计进度情况（本周单元设计的进度情况）】

这周写组合模式（Compose Mode）。这个功能允许多个媒体源（摄像头+屏幕+文件）在一个画布上同时显示，类似导播台的效果。底层实现是：从各源的FrameBus订阅YUV帧，在一个后台线程里把多路YUV帧合成一张大画布——摄像头放左上、屏幕放右上——然后用sws_scale做缩放和格式转换，合成后推到编码器。UI层用QMdiArea管理子窗口，每个源一个窗口，可以自由拖拽、缩放、调整层级。

【指导教师指导情况】

导师建议画布的合成逻辑用纯CPU做就好，不要上OpenCL/CUDA，因为毕业设计的重点是跑通流程，性能优化可以放到“后续改进”里写进论文。另外提醒打包的时候MDI区域的坐标要跟YUV合成坐标对应起来。

【遇到的疑难问题】

多路YUV合成的时候，如果两路视频的分辨率不一样（比如4K屏幕+1080p摄像头），sws_scale缩放+合并的性能掉得厉害。实测三路1080p一起合成到4K画布上只有15fps左右，远远达不到流畅观看的标准。

【疑难问题的解决途径】

优化了三处：一是合成前先做一次分辨率归一化，各路统一缩放到目标尺寸的1/2再合成；二是sws_scale的flags改用SWS_FAST_BILINEAR；三是把合成操作从主线程挪到后台线程，主线程只负责交换最终结果。优化后能稳在25fps，基本可用了。

第十二周

【毕业设计进度情况（本周单元设计的进度情况）】

这周重心转到服务端。用Go写了Gateway网关服务（gateway/main.go），纯标准库HTTP服务没有依赖第三方框架，监听9000端口。实现了几个核心接口：POST /api/v1/streams/start（注册流）、GET /api/v1/streams/resolve?app=xxx&stream=yyy（解析播放地址）、GET /api/v1/streams（列所有流）、POST /api/v1/streams/{id}/stop（停流）、GET /healthz（健康检查）等。流信息用sync.RWMutex保护的map存内存。还解决了Linux下多网卡的host选择问题——多个网卡时自动找到最合适的IP对外发布。

【指导教师指导情况】

导师看了API设计后提出resolve接口应该支持按协议返回不同URL（RTMP/HTTP-FLV/HLS），因为桌面端和手机端需要的协议不一样。另外建议加上CORS中间件，后面手机端可能会跨域请求。

【遇到的疑难问题】

在Windows上开发的时候，Go的net.Listen绑定0.0.0.0没问题，但是放到Linux服务器上有多个网卡（eth0, docker0, lo等），自动选的IP经常是docker0的172网段，外面根本访问不了。

【疑难问题的解决途径】

写了一个智能选host的函数：先过滤掉回环地址和docker虚拟网卡，优先选/24子网下能ping通网关的地址，再不行就选第一个有路由的非虚拟网卡IP。这个逻辑放在getSmartHost函数里，在resolve接口返回URL时调用。

第十三周

【毕业设计进度情况（本周单元设计的进度情况）】

这周写kernel-console和Electron管理界面。kernel-console用Go写的启动器，自动检测端口占用情况（RTMP默认1935、HTTP默认8080），动态分配空闲端口，然后修改ZLMediaKit的config.ini模板生成运行时配置文件，再以子进程方式启动ZLM和Gateway，统一管理生命周期。Electron UI（980x720的窗口）用原生JS+HTML写了一个控制面板：可以创建流、显示推流/拉流地址、查看ZLM和Gateway的日志。主进程通过IPC跟渲染进程通信，启动和停止服务核心。

【指导教师指导情况】

导师试跑了一遍Electron界面后觉得端口自动检测这个功能很好，手动配端口太容易冲突了。但建议把启动失败的提示做得更友好一点，比如端口全被占用时提示用户检查防火墙，而不是只报一个"启动失败"。

【遇到的疑难问题】

Electron打包的时候，node_modules里带了一大堆dev依赖和平台相关的二进制，打出来的包有300多MB，太夸张了。ZLM的二进制文件也有100多MB，两边加起来快500MB了，作为一个小工具来说体积太大。

【疑难问题的解决途径】

用electron-builder的files配置精确指定要打包的文件，排除了devDependencies里不需要的包，包体积直接砍了一半多。ZLM这边经过权衡决定用SRS的docker镜像作为替代方案，本地开发用ZLM，正式部署推荐用docker-compose一键启动。

第十四周

【毕业设计进度情况（本周单元设计的进度情况）】

这周开始写手机端。用Flutter搭了一个单页App（lib/main.dart一个文件写了差不多1400行），主要是功能集中不需要拆分太多文件。核心功能：一个16:9的视频播放区，用media_kit（底层是libmpv）来渲染。模式用SegmentedButton在"服务模式"和"直连模式"之间切。服务模式下输入Gateway地址、应用名、流名，选协议（HLS/HTTP-FLV/RTMP），点拉流后先调Gateway的resolve接口拿播放地址，再用media_kit播放。直连模式直接输入URL播放。所有输入参数用SharedPreferences持久化，下次打开自动回填。

【指导教师指导情况】

导师建议手机端优先用HLS协议，因为Android和iOS对HLS的原生支持最好，不依赖第三方解码器。另外media_kit在Android上有一些兼容性问题，建议多测几个Android版本。

【遇到的疑难问题】

media_kit在播放RTMP流的时候偶尔会闪退，查日志发现是libmpv在收到不完整的RTMP数据包时会触发断言失败直接崩溃，Flutter这边抓不到异常，应用直接没了。

【疑难问题的解决途径】

查了media_kit的GitHub issues，发现这是个已知问题，社区暂时没有完美的修复方案。解决方法是把协议优先级改了——默认选HLS，HTTP-FLV次之，RTMP放最后。桌面端推流到ZLM后ZLM会自动把RTMP transmux成HLS和HTTP-FLV，手机端用HLS就行。

第十五周

【毕业设计进度情况（本周单元设计的进度情况）】

这周把手机端收尾。加了一个内嵌日志面板——用终端风格的深色背景+彩色日志（info绿/warn黄/error红），200条环形缓冲，支持一键复制和清空，调试的时候很好用。播放器控制栏做了简化（毕竟是直播流不是点播，不需要进度条），只有播放/暂停、音量、全屏三个按钮，保持界面清爽。还写了PowerShell打包脚本——一键生成APK、AAB、Windows ZIP，自动算checksum。桌面端这边也完善了深色/浅色主题切换和自定义强调色。

【指导教师指导情况】

导师看了手机端的日志面板后觉得这个设计很实用，调试的时候不用连电脑看logcat。建议在桌面端也加一个类似的日志面板，特别是在推流失败的时候能直观看到错误信息，而不是只靠控制台输出。

【遇到的疑难问题】

Flutter的SharedPreferences在Android上第一次启动时会有几百毫秒的延迟，导致界面刚出现时输入框还是空的，然后突然跳出上次保存的值，体验不太好，有种闪一下的感觉。

【疑难问题的解决途径】

在initState里先用同步方式读取SharedPreferences的缓存值（它内部其实维护了内存缓存），设置初始默认值，然后再异步load最新值覆盖。这样界面一渲染就有上次的值，用户不会看到内容从空白到突然出现的闪烁。

第十六周

【毕业设计进度情况（本周单元设计的进度情况）】

这周实现了AI图像分析功能。在桌面端加了ImagePoolService——扫描截图保存目录，生成缩略图，用QFileSystemWatcher监听新文件自动刷新侧边栏。侧边栏显示缩略图网格，支持按日期/名称/大小排序，右键可以打开或删除。AiChatDialog是一个左右分栏的对话框——左边预览选中的图片，右边是对话气泡（用户和AI气泡颜色可配置），通过SSE流式接收AI回复，文字逐个打出来，带闪烁光标，体验类似ChatGPT。底层对接OpenAI兼容的视觉模型API。

【指导教师指导情况】

导师对AI功能持保留态度，认为作为亮点功能可以加但不要喧宾夺主。建议在论文里把AI分析定位为"扩展应用场景"，重点还是流媒体系统本身的核心能力。

【遇到的疑难问题】

SSE流式响应有时候会中断，特别是上传的截图分辨率比较大（4K），AI分析时间超过30秒后Nginx或者代理层可能会超时断开连接。小图片没问题，大图片基本上每次都会在中途断掉。

【疑难问题的解决途径】

在发送前先把图片压缩到1080p以下，用QImage的scaled做了等比缩放，这样上传数据量小很多，AI响应也更快更稳。同时在AiService里加了断线重连逻辑——SSE断开后自动重试一次，因为模型端处理过的token不会重复计费。

第十七周

【毕业设计进度情况（本周单元设计的进度情况）】

最后一周主要做测试、打包和论文。桌面端用CPack成功打出了Windows的NSIS安装包和Linux的DEB包，写了一个PowerShell脚本把windeployqt、FFmpeg dll、配置文件全部自动收集打包，一条命令就完成。手机端用build_release.ps1打了APK和AAB。整个系统打通跑了一遍：桌面端采集摄像头画面 -> 推RTMP到服务端ZLM -> Gateway注册流 -> 手机端调resolve获取HLS地址 -> 播放。延迟大概在2-3秒左右，属于直播场景可接受的范围，后面可以继续优化。

【指导教师指导情况】

导师最后帮我把系统整体过了一遍，提出了几个论文写作的建议：重点强调分层架构和零拷贝设计这两个创新点；把AI和主题系统作为锦上添花的扩展功能；测试数据要有对比，这样更有说服力。

【遇到的疑难问题】

安装到一台全新的Windows 11机器上测试时，发现缺少VC++运行时库，程序启动直接报0xc000007b错误无法运行。检查后发现NSIS打包的时候没有把vcredist带进去，新机器上面没有这些运行库就很尴尬。

【疑难问题的解决途径】

在CPack的NSIS配置里加了VC++运行时库的捆绑安装，打包脚本里多了一个步骤自动检测系统VC++版本并把安装器放进最终安装包。同时在README里写了手动安装VC++运行时的说明作为备选方案，双保险。

总17周，覆盖从选题调研到系统集成测试的完整开发周期。